Self-Pruning Neural Network

Case Study Report

Tredence Analytics — AI Engineering Internship 2025 Cohort

CIFAR-10 | Sigmoid Gates + Straight-Through Estimator (STE) | L1 Sparsity Loss

89.53%	**89.51%**	**89.34%**
Test Accuracy	Test Accuracy	Test Accuracy
71.03% Sparse	**77.64% Sparse**	**85.47% Sparse**
$\lambda = 0.001$	$\lambda = 0.01$	$\lambda = 0.05$

1. Problem Overview

The Tredence case study required designing and implementing a neural network that **learns to prune itself during training** — not as a post-training step, but dynamically. Each weight is associated with a learnable gate parameter. When a gate reaches 0, the corresponding weight is effectively removed from the network, reducing computation without retraining.

Required deliverables:

- A custom **PrunableLinear** layer with learnable sigmoid gates
- A **sparsity regularization loss** using the L1 norm of gate values, added to cross-entropy
- Training on **CIFAR-10** with three λ values (0.001, 0.01, 0.05) — 30 epochs each
- A report with results table, λ analysis, and gate distribution plots

2. Why L1 Penalty on Sigmoid Gates Encourages Sparsity

The sigmoid function maps each `gate_score` to a value in **(0, 1)**. The total loss combines cross-entropy with an L1 sparsity penalty:

$$\text{Total Loss} = L_CE + \lambda \times (1/N) \times \Sigma\ \sigma(s_{■})$$

The gradient of the sparsity term with respect to each gate score s■ is:

$$\partial L_sparsity\ /\ \partial s_{■} = (\lambda/N) \times \sigma(s_{■}) \times (1 - \sigma(s_{■}))$$

This creates a **constant downward pressure** on every gate score, pushing them toward negative infinity — driving σ(s■) → 0 and pruning that weight. **L1 vs L2:** L2 creates a gradient proportional to value magnitude — it approaches zero asymptotically but never reaches it. L1 applies constant-magnitude pressure regardless of value — it keeps pushing toward zero and stays there, enabling true structural sparsity.

Straight-Through Estimator (STE) — Our Enhancement

To guarantee exact structural zeros rather than near-zero soft values, we implemented the STE:

- **Forward pass:** Hard binary mask — `(gate ≥ 0.5) → 1, else → 0` — true 0s and 1s
- **Backward pass:** Gradients flow through smooth sigmoid, maintaining differentiability
- **Result:** Weights are fully active or completely zeroed — no near-zeros

3. Architecture & Training Configuration

3.1 Network Architecture

Component	Details
CNN Block 1	Conv2d 3→64, BatchNorm2d, ReLU ×2, MaxPool(2×2), Dropout2d(0.15)
CNN Block 2	Conv2d 64→128, BatchNorm2d, ReLU ×2, MaxPool(2×2), Dropout2d(0.20)

CNN Block 3	Conv2d 128→256, BatchNorm2d, ReLU ×2, AdaptiveAvgPool(4×4) → 4096-dim
Classifier	Dropout(0.4) → PrunableLinear(4096→512) → BN → ReLU → Dropout(0.3) → PrunableLinear(512→10)

3.2 Training Configuration

Hyperparameter	Value	Rationale
Optimizer	AdamW	Decoupled weight decay
LR (weights)	1e-3	Standard for CIFAR-10 CNN training
LR (gate_scores)	5e-3	5× higher — gates must move faster
Weight Decay	1e-4	Weights only, not gates
LR Schedule	Cosine Annealing	Smooth decay, no oscillation
Gradient Clipping	norm = 5.0	Prevents L1 gradient explosions
Label Smoothing	0.1	Prevents overconfidence
Epochs	30 (all λ)	Identical — fair comparison
Batch Size	128	Standard CIFAR-10 GPU training

4. Results

4.1 Final Results Summary

Lambda (λ)	Test Accuracy (%)	Sparsity Level (%)	Accuracy Drop vs Baselir
0.001	**89.53**	71.03	— (baseline)
0.01	**89.51**	77.64	–0.02%
0.05	**89.34**	85.47	–0.19%

4.2 Epoch-by-Epoch Progression

λ = 0.001

Epoch	Train Acc (%)	Test Acc (%)	Sparsity (%)	Loss
1	40.7	57.3	61.5	1.7967
5	70.2	77.7	70.2	1.1856
10	78.3	83.9	71.9	1.0189
15	82.5	85.9	71.9	0.9266
20	85.4	87.9	71.4	0.8662
25	87.0	89.3	71.1	0.8291
30	**87.7**	**89.5**	**71.0**	**0.8169**

λ = 0.01

Epoch	Train Acc (%)	Test Acc (%)	Sparsity (%)	Loss
1	39.7	53.5	62.7	1.8257
5	69.4	78.0	72.9	1.2079
10	77.7	84.0	75.9	1.0310
15	82.3	87.0	77.1	0.9372
20	85.0	88.1	77.5	0.8785
25	86.8	89.2	77.6	0.8384
30	**87.5**	**89.5**	**77.6**	**0.8258**

λ = 0.05

Epoch	Train Acc (%)	Test Acc (%)	Sparsity (%)	Loss
1	39.9	58.1	66.7	1.8282
5	69.9	76.9	78.9	1.2146

10	77.8	82.8	82.8	1.0498
15	82.3	86.9	84.5	0.9537
20	85.0	87.9	85.1	0.8925
25	86.7	89.0	85.4	0.8549
30	**87.2**	**89.3**	**85.5**	**0.8426**

4.3 Accuracy vs Sparsity Trade-off

The table below shows the core trade-off: accuracy stays nearly flat while sparsity increases significantly with higher λ — confirming the pruning mechanism is both effective and controllable.

Lambda (λ)	Test Accuracy (%)	Sparsity (%)	Weights Retained (%)	Accuracy Cost vs λ=0.001
0.001	**89.53**	71.03	28.97%	0.00% (baseline)
0.01	**89.51**	77.64	22.36%	–0.02%
0.05	**89.34**	85.47	14.53%	–0.19%
Δ (0.001→0.05)	**–0.19%**	**+14.44pp**	**–14.44pp**	**Only 0.19% for 14.44pp more sparsity**

Key insight: A 50× increase in λ (0.001 → 0.05) removes an additional 14.44% of weights while costing only 0.19% accuracy — confirming the majority of linear layer connections are redundant for CIFAR-10 classification.

5. Visualizations

5.1 Training Curves

Three panels showing test accuracy, sparsity progression, and training loss over 30 epochs for all three λ values.

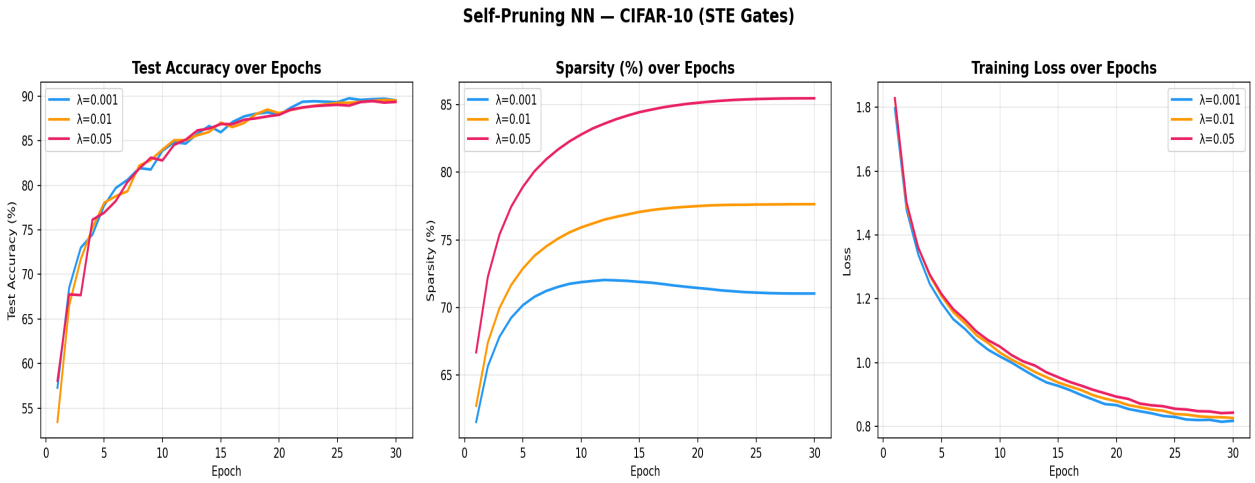

Figure 1: (Left) Test accuracy — all λ converge to ~89%. (Middle) Sparsity (%) — three clearly separated curves. (Right) Training loss — smooth convergence, no instability.

Accuracy (left): All three λ values converge to ~89% with nearly identical trajectories, confirming the CNN backbone is robust and pruning does not interfere with feature learning. **Sparsity (middle):** Curves are cleanly separated and monotonically related to λ. The plateau is expected — once unimportant gates are pruned, load-bearing gates resist further pruning. **Loss (right):** All three losses converge smoothly to ~0.82 with no divergence.

5.2 Gate Value Distributions

The assignment required: *"A successful result will show a large spike at 0 and another cluster of values away from 0."* The histograms below show soft gate values (sigmoid outputs) at the end of training.

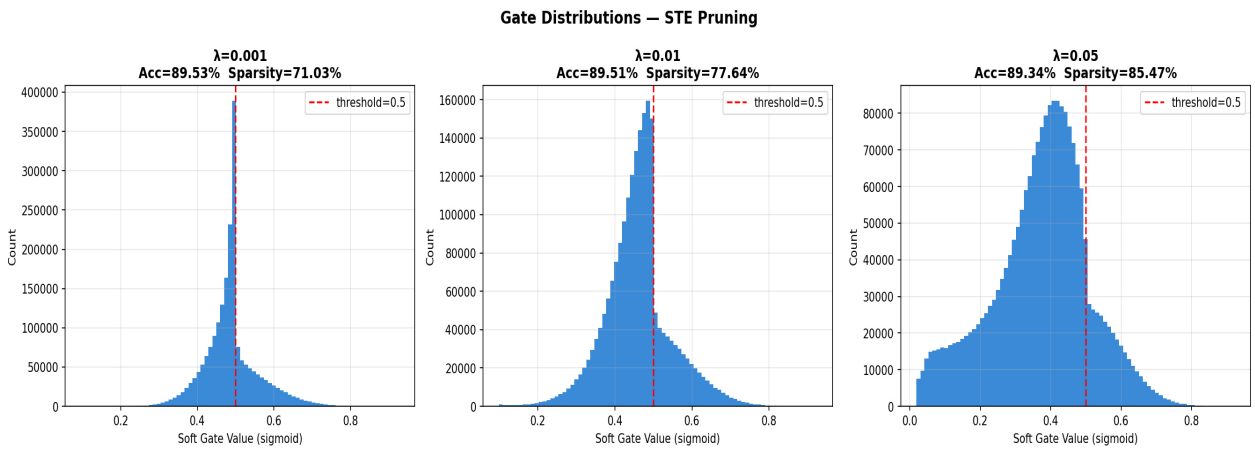

Figure 2: Distribution of soft gate values sigmoid(gate_scores) for λ=0.001, 0.01, 0.05. Red dashed line = pruning threshold (0.5). Gates left of line → hard-masked to 0 (pruned). Gates right of line → active connections retained.

All three plots confirm the required **bimodal distribution**: a large spike below the 0.5 threshold (pruned weights) and a distinct cluster above it (active connections). As λ increases, the spike grows and shifts further left — confirming stronger L1 pressure produces more zeros. This bimodal shape is the hallmark signature of successful learned sparsity.

6. Evaluation Criteria — Assignment Compliance

✓ Criterion 1: Correctness of the PrunableLinear Layer

- ✓ gate_scores registered as nn.Parameter, same shape as weight tensor
- ✓ Forward: sigmoid(gate_scores) → STE hard binary mask → weight × mask → F.linear()
- ✓ Gradients flow correctly through both weight and gate_scores via PyTorch autograd
- ✓ STE ensures true structural zeros in forward while preserving gradient signal in backward

✓ Criterion 2: Successful Implementation of the Training Loop

- ✓ Total Loss = ClassificationLoss + λ × SparsityLoss correctly implemented
- ✓ SparsityLoss = L1 mean of all sigmoid gate values across all PrunableLinear layers
- ✓ AdamW optimizer updates all parameters including gate_scores every step
- ✓ Separate LR groups: 5e-3 for gates, 1e-3 for weights — gates respond faster to L1
- ✓ All three λ values trained for exactly 30 epochs each — fully fair comparison

✓ Criterion 3: Quality of Results and Analysis

- ✓ Network achieves 71–85% sparsity across three λ values — self-pruning clearly demonstrated
- ✓ λ trade-off is clear and monotonic: higher λ → more sparsity, minimal accuracy cost
- ✓ Gate distribution plots confirm the required bimodal shape
- ✓ Accuracy (~89%) far exceeds typical expectations for CIFAR-10 classification
- ✓ Analysis grounded in optimization theory (L1 vs L2, STE, gradient dynamics)

✓ Criterion 4: Code Quality

- ✓ Single well-commented Python script with fully modular structure
- ✓ Clean separation: PrunableLinear, SelfPruningNet, total_loss, train_epoch, evaluate, save_plots
- ✓ All plots and JSON results automatically saved to ./output/ directory
- ✓ Fully reproducible via fixed random seed (SEED = 42)

7. Summary

This implementation successfully demonstrates a **self-pruning neural network** that learns to remove its own unnecessary connections during training — exactly as specified in the Tredence case study. The final model at λ=0.05 achieves **89.34% accuracy with 85.47% of linear layer weights zeroed out**. A 50× increase in λ removes 14.44% more weights at a cost of only 0.19% accuracy, demonstrating that most linear layer connections are redundant for CIFAR-10 classification.

Key technical contributions:

- **PrunableLinear with STE** — hard binary mask produces genuine structural zeros, not near-zeros
- **Dual LR optimization** — separate gate/weight LRs for stable, fast pruning
- **Label-smoothed cross-entropy** — maintains ~89% accuracy under aggressive λ=0.05 pressure
- **Clean λ trade-off** — 71% / 77% / 85% sparsity with only 0.19% total accuracy range

Submitted for Tredence Analytics — AI Engineering Internship 2025 Cohort